Algoritmos e Sistemas Distribuídos

Filipe Colla David and Victor Ditadi Perdigão
NOVA Laboratory for Computer Science and Informatics (NOVA LINCS)
and
Departamento de Informática
Faculdade de Ciências e Tecnologia
Universidade NOVA de Lisboa
(Based on notes produced by João Leitão)

December 20, 2024

Abstract

In the world of distributed systems, ensuring consistent replication among machines is crucial to achieve better performance, scalability and fault tolerance, this replication is also known as state machine replication (SMR). There are multiple ways to solve SMR, in this report will be discussing an ABD based SMR implementation and Multi-Paxos. The goal of the report is to explore each of these approaches, point out their benefits and drawbacks. Identifying these tradeoffs is crucial for the development of robust and efficient systems that are adequate to solve a specific problem.

1 Introduction

As demonstrated by FLP[**?**] solution, achieving distributed consensus in a asynchronous environment with one faulty process is impossible, however, we can relax some properties of the consensus problem in order to have distributed consensus that works most of the times. In this report will be exploring different ways to relax the termination property of distributed consensus in order to have a practical consistent state replicated system across nodes using an ABD[**?**] based state machine protocol and Multi-Paxos[**?**].

The state replication described in this report consists of a replicated key-value map and the membership of the system, the goal of this project is to ensure consistent replication of these states across each replica[1], so that a client requesting an read/write operation will be able to ensure that his write requests are atomically replicated in the system and that his read request will always return the last value updated in the system.

[1] A replica is a node participating in the quorum decisions of the system

Multi-Paxos is a state machine replication protocol that admits and handles crash failures automatically, ABD by itself is just a quorum based system that guarantees atomic read-writes operations on a single registry, in order to implement state machine replication, some changes are needed.

The implementation of these protocols will be done in Java using the Babel Framework[**?**].

2 Related Work

2.1 Babel Framework

The Babel framework is a Java-based framework developed in the NOVA LINCS (Nova Laboratory for Computer Science and Informatics)[**?**], designed to simplify the development of distributed, fault-tolerant, high-performance protocols. It abstracts the low-level complexities such as the communication management timeouts, and concurrency, allowing developers to focus on core protocol logic. Babel uses an event-driver model where protocols are implement as independent event-based state machines that process timers, messages and requests, and emit notifications.

In this report, the protocols were implemented using this framework, the nomenclature for the Pseudocode and explanations are in accordance with be the ones used in its respective paper.

2.2 Multi-Paxos

In the paper "Paxos Made Simple"[**?**](implicitly describing Multi-Paxos as an optimization), Leslie Lamport explores how to optimize the Paxos consensus protocol for practical state machine replication. The protocol became widely recognized after its implementation by major companies like Google, and has since become a cornerstone

for building fault-tolerant distributed systems, being particularly efficient for maintaining consistent state across multiple replicas.

The key insight of Multi-Paxos is that in a sequence of consensus decisions, such as in state machine replication, it's inefficient to run the complete Paxos protocol for each decision. Instead, after a leader is elected, subsequent consensus decisions can skip the prepare phase, reducing the message exchanges from 4 to 2 rounds. This optimization proves especially valuable in geo-distributed settings where network latency is significant.

The protocol divides consensus into ordered slots, where each slot represents one decision in the sequence. While a leader remains stable, it can propose values directly for consecutive slots using only the accept phase of Paxos. If the leader fails or becomes unreachable, a new leader is elected through a complete Paxos instance, ensuring all previously accepted values are preserved before resuming normal operation.

Multi-Paxos can be enhanced to handle membership changes automatically by treating them as special values that must be agreed upon through the consensus sequence. The protocol ensures that membership changes are totally ordered with respect to other operations, maintaining consistency even as the set of participating replicas evolves. These mechanics will be detailed in the Implementation(3.1) section of this report.

2.3 ABD

In the paper "Sharing Memory Robustly in Message-Passing Systems"[**?**] (also known as ABD - H. Attiya, A. Bar-Noy, D. Dolev), the authors explore the implementation of sharing memory in message-passing systems, by presenting emulators that translate shared-memory algorithms to message-passing models, implementing wait-free, atomic, single-writer multi-reader registers in unreliable, asynchronous networks. This approach takes advantages of quorum based systems in order to achieve replication of a single register, it does so by ensuring that every write is propagated through the system, and that every read returns the most recent value in the system.

It's easily adapted to multiple registers, by running several instances of ABD in parallel identified by a key, additionally we can also run instances of ABD to automatically manage the membership of the system.

By changing the original protocol to account for multiple registers and handle membership change, we can achieve a SMR similar to Multi-Paxos(2.2), where membership is automatically managed via a series o mechanisms. These mechanisms will be expanded in the Implementation(3.2) section of this report

3 Implementation

3.1 Multi-Paxos based SMR

The implementation consists of two main components: a **State Machine Replication (SMR)** protocol and an **Agreement** protocol based on Multi-Paxos. The SMR protocol manages the replicated key-value store state and receives operation requests from HashApp or operation messages from other nodes, while the Agreement protocol ensures consistent ordering of operations across replicas through consensus. The system assumes that before reaching a minimum quorum (3 replicas), the first node will be the leader and any new node join request will be immediately decided by the leader without running consensus. After reaching the minimum quorum size, the SMR protocol relies on the leader to order operations, which are then committed through Multi-Paxos consensus to ensure all replicas execute operations in the same order. A heartbeat mechanism is used to detect leader failures, triggering leader election when necessary through the Agreement protocol.

3.1.1 Leader Election

The leader election mechanism in our Multi-Paxos implementation is built around a heartbeat-based failure detection system. The protocol maintains two main timers: a leader monitoring timer that runs on follower nodes and a heartbeat sending timer that runs on the leader node.

Leader nodes are configured to send heartbeat messages every `LEADER_TIMEOUT/2` milliseconds (where `LEADER_TIMEOUT` is set to 5000ms in the implementation). This is done through a periodic timer that triggers a heartbeat message to be sent to all other replicas in the membership:

```
if isLeader then
    heartbeatMessage ← new HeartbeatMessage()
    for all host ∈ membership do
        if !host.equals(self) then
            sendMessage(heartbeatMessage, host)
        end if
    end for
end if
```

Follower nodes maintain a boolean flag `receivedHeartbeat` that is set to true whenever they receive a heartbeat message from the current leader. A periodic timer running every `LEADER_TIMEOUT` milliseconds checks this flag. If no heartbeat was received during this period, the follower initiates a leader election by starting a new instance of Paxos consensus with a special leader election operation:

```
if !isLeader and !receivedHeartbeat then
    startLeaderElection()
    // Create new Paxos instance for leader election
    // Propose self as the new leader
end if
receivedHeartbeat ← false
```

The leader election itself is conducted through a complete Paxos instance, where the proposed value is the identity of the new leader. This ensures all replicas agree on the new leader even in the presence of multiple concurrent leader elections.

When the Agreement protocol completes the Paxos consensus for leader election, each node's Agreement protocol will notify its local SMR protocol through a `LeaderDecidedNotification`. This notification triggers the following state changes in the SMR protocol:

- If the local node is the new leader, sets its `isLeader` flag to true and starts the heartbeat timer
- Updates its `currentLeader` reference
- If not the leader, establishes a connection to the new leader

By using Paxos consensus for leader election, the system ensures that even with multiple nodes detecting leader failure simultaneously and proposing themselves as leaders, only one leader will be elected and acknowledged by all functioning replicas in the system.

3.1.2 Membership Management

The membership management in our implementation is responsible for handling new nodes joining the system and maintaining a consistent view of the active replicas across all nodes. The system maintains a `LinkedList<Host>` to track current members, where each `Host` represents a replica's network address and port.

When a new node wants to join the system, it sends a join request to its contact node. The behavior for processing this request varies depending on whether the system has reached minimum quorum (3 replicas):

Before minimum quorum:

- The leader can immediately accept new nodes without running consensus
- The leader updates its membership list and sends a `StateTransferMessage` to the joining node
- The joining node becomes ready to participate after processing the `StateTransferMessage`

After minimum quorum:

- The contact node forwards the join request to the current leader
- The leader creates a special join operation and initiates a Paxos instance
- Upon consensus completion, the leader sends a `StateTransferMessage` to the joining node
- The joining node becomes ready to participate after processing the `StateTransferMessage`

The `StateTransferMessage` contains the current membership list and the system state. When a node receives this message, it updates its local state and becomes ready to participate in the system:

```
Upon ReceivedStateTransferMessage(msg, host) do
    if !msg.getLeader().equals(self) then
        this.membership ← msg.getMembership()
        this.applicationState ← deserializeState(msg.getState())
        this.readyOperationInstance ← true
    end if
end Upon
```

Each membership change is treated as a special operation that must be processed before any subsequent operations can be executed, ensuring consistency in the membership view across all replicas.

3.1.3 Consensus Implementation

The consensus implementation optimizes for the common case of a stable leader while maintaining safety during leader changes:

1. **Normal Operation (Stable Leader):**
   - Leader skips Prepare phase and directly sends AcceptPaxosMessage to all replicas
   - Acceptors respond with LearnPaxosMessage
   - Decision reached after quorum of Learn messages
2. **Leader Recovery:**
   - New leader must start complete Paxos instance with Prepare phase
   - Sends PrepareMessage with higher ballot number
   - Collects Promise responses from quorum
   - Proceeds with Accept phase after quorum reached
3. **Decision Processing:**
   - Upon receiving a quorum of accepts, processes decision based on operation type

- For leader election operations, updates leadership state and starts heartbeat if elected
- For join operations, updates membership list with new node
- For regular operations, triggers notification to SMR for execution
- All decisions update internal state and clean up operation metadata

The implementation maintains all necessary state for each Paxos instance in a `PaxosInstance` class, which tracks:

- Ballot numbers
- Promise and accept quorums
- Instance type (REGULAR, JOIN, or LEADER-ELECTION)
- Proposed values

When the Agreement protocol reaches consensus on an operation, it triggers the appropriate notification to the SMR protocol, which then executes the operation and updates its state.

3.1.4 Operation Processing

Operation processing in our Multi-Paxos based system begins when HashApp generates operation requests. These requests follow different paths depending on whether they are received by a leader or non-leader replica. When a replica receives an OrderRequest from HashApp, it first checks if it's ready to process operations and if the operation hasn't been previously decided: If the replica is not the leader, it forwards the operation to the current leader via an OperationMessage:

```
if !isLeader then
    OperationMessage msg = new OperationMessage(
      request.getOpId(), self,
      nextOperationInstance++,
      request.getOperation())
    openConnection(currentLeader)
    sendMessage(msg, currentLeader)
    return
end if
```

If the replica is the leader, it proposes the operation through the Agreement protocol:

```
ProposeRequest proposeRequest = new ProposeRequest(
  nextOperationInstance++,
  request.getOpId(),
  request.getOperation())
sendRequest(proposeRequest, AgreementProtocol.PROTOCOLID)
```

When an operation is decided, the Agreement protocol notifies the SMR through a DecidedNotification. Upon receiving this notification, the SMR:

Verifies the operation hasn't been already decided Increments the next operation instance number Updates its membership list Executes the operation if there is payload Records the operation as decided

In case of leadership changes, operations in progress may need to be restarted under the new leader. The system ensures safety by tracking decided operations to prevent duplicate execution.

3.2 ABD based SMR

The implementation of state machine replication using quorums is essentially multiple ABD instances running in parallel with different objectives guaranteeing serialisability of each individual key or each membership change in the system. These objectives are defined in our implementation as **Operations** (Read Write Operation and Membership Operation). This implementation also assumes that the system starts with, at least, three initial replicas (initial membership), but can handle memberships joining afterwards or leaving. Additionally, the system works as intended as long as three replicas remain in it.

3.2.1 Initialization

The initialization of each ABD replica is described in the appendix A, Algorithm 1. Upon starting, each replica starts initializes it's own state, that consists of:

- `membership`: An HashSet, used to maintain the current membership of the system
- `currentlyAlive`, `pendingMembership`, `membershipTag`: Helper data structures (HashSets) for maintaining the membership of the system. (See 3.2.4).
- `val`: Holds the key-value map to be replicated.
- `tag`: A key-value map that holds the tag associated with each key (See 3.2.3)
- `inProgressOperations`: Holds the set operations that are currently in progress at a given instance.

- `pendingOperations`: A Queue that stores operations received by the application, but are yet to be executed.
- `opSeq`: An HashSet that holds the current number of operations of this process.

If a replica integrates the initial membership of the system, it just adds the other replicas to the the membership, and updates it's `ready` state to `true`, meaning that it's immediately available to execute any request made by either the application or other replicas. Otherwise, it joins by requesting to a contact to join the system, then, it has to wait until it's integrated to start participating in the quorum. In the implementation we use a boolean to represent the `ready` state of replica, however in the pseudocode this state is verified by checking weather the `replicas` set was not initialized (is null or bottom).

3.2.2 Operations Queues

The Operation object serves as a wrapper for each individual request. It's defined as an abstract class, implemented by the `ReadWriteOperation` class and the `MembershipOperation` class. The `ReadWriteOperation` class represents any Read or Write request made by the application, and the `MembershipOperation` class represents a membership change requested by a replica. When receiving these requests, they are placed in the `pendingOperations` queue. This queue is then accessed repeatedly, with a predefined interval by a timer method event that processes the first operation in the queue and execute it's respective request.

In order to avoid having multiple operations (in the same replica) executing for the same key, `inProgressOperations` holds the operations that are currently waiting to terminate. Upon starting an operation, the replica verifies if the `inProgressOperations` hashset already contains an operation for that key, if so, it wait's until that operation is completed.

Effectively, we can have has many concurrent Read/Write operations as there are keys, however, to avoid overloading the CPU, we've created a mechanism that limits the amount of concurrent operations, by limiting the maximum size of the `inProgressOperations` set.

The objects and methods described in this subsection are not represented in the pseudo code as they're not part of the algorithm itself, but an improvement to handle multiple concurrent requests on different replicas.

3.2.3 Read Write Operations

The description of these operations follows the pseudocode available in the appendix A Algorithm 2.

Write Operation: The replica that receives the Write request (replica one), issues a request to all the other replicas to read the tag associated with a specific key, if the key is yet to be seen by any of the other replicas, it returns tag with a negative value. Replica one, upon reaching a quorum size of responses from the tags requested, it gets the maximum value from those tags, increments it, and sends a message to the other replicas to update their value and the tag associated with that key. The other replicas, when receiving this message, return an ack. Replica one, upon receiving these Acks and reaching a quorum, notifies the application that the write was complete.

Read Operations: The replica that receives the Read request for a specific key (replica one), requests a read for that key from all the other replicas in the system, from which they send the value and it's respective tag. When replica one receives a quorum size of responses, selects the value with the higher tag and, sends a write request to the other replicas, ensuring that this value is propagated. Upon receiving the write request for the new value, the other replicas update their state, and return an Ack. Replica one, upon receiving enough Acks to build a quorum, notifies the application that the read operation was complete and returns the most up to date value.

3.2.4 Membership Management

The Membership Management consists of either a replica joining or leaving the system. In contrast with the Read and Write requests, these requests are received from other replicas that detect changes in the membership, and they're all placed in the first position of the queue, additionally, they only start executing when there are no read or write requests in progress. The pseudo code for the Membership Management is described in the appendix A, Algorithm 3 and Algorithm 4.

Joining the system: When a replica is trying to join the system (replica two), it sends a Join Request to it's contact (replica one), that adds a request to add a new replica to the first position of the `pendingOperations` queue. Replica one, upon starting the operation, it sends a message for other replicas to send their membership tags, upon receiving enough replies to make a quorum, replica one gets the biggest tag, increments it, and sends it alongside with the replica to be added to the membership. The other replicas, upon receiving the information of the new replica, place it in the `pendingMembership` set, and return an Ack to replica one. When replica one has enough Acks to make a quorum, sends a message with the current membership information to replica two, that upon receiving that information, set it's `ready` state to true, sends a message to all the replicas in the membership, which in turn, remove this replica from the `pendingMembership` set and add it to the

`membership` set. Replica two is now ready to start participating in the quorum. Since reads and writes are propagated, there's no need to share state, because the replicas will always quorum and propagate the most recent value.

Leaving the System: Each replica periodically triggers a timer, configurable to a specific interval, that checks on all the other replicas. It does so, by sending a ping to every replica, then adds every replica that responds to the `currentlyAlive`. When the timer triggers again, it checks if there are any replicas from the `membership` set missing in the `currentlyAlive` set, if so, adds the operation to remove the missing replicas to the first position of the `pendingOperations` queue.

The execution of this operation is the mostly the same from the operation of adding a new replica, however, when the other replicas receive the information to remove the replica, the replica gets removed immediately and an ack is sent. The replica who requested the Remove operation, waits for a quorum (already with one less element) and removes the replica as well.

The implementation of this membership management protocol is, effectively, a an ABD write operation.

4 Experimentation and Results

The experimental evaluation was conducted in a distributed environment using four nodes, one as the client and three servers. The test scenario consisted of executing 5,000 operations with a balanced mix of reads and writes. We compared three different consensus protocols: basic Paxos, Multi-Paxos, and ABD (Attiya, Bar-Noy, Dolev). All tests were performed with consistent parameters including quorum size (2 out of 3 nodes) and network conditions. Additionally, for ABD, the timer that executes `pendingOperations` was set to 5 μs. The value of this interval changes drastically results of the operations, increasing sthe value leeds to higher response times, lowering increases the probability of synchronization issues (see Section 5.2)

4.0.1 Overall Throughput

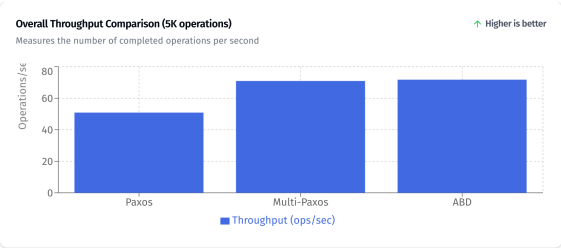

Figure 1: Overall Throughput Comparison (5K operations)

The throughput analysis reveals significant performance differences among the protocols. Multi-Paxos and ABD demonstrate similar throughput capabilities (70.93 and 71.77 ops/sec respectively), both substantially outperforming basic Paxos (50.89 ops/sec). This performance gap can be attributed to the reduced message complexity in both Multi-Paxos and ABD compared to basic Paxos. The similarity between Multi-Paxos and ABD throughput suggests that both implementations of the protocols effectively optimize their communication patterns for sustained operation.

4.0.2 Latency Distribution

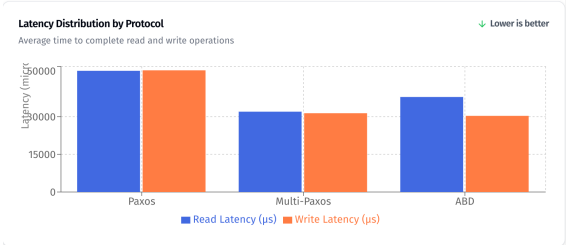

Figure 2: Latency Distribution by Protocol

Latency measurements show distinct characteristics for each protocol. Basic Paxos exhibits the highest latencies (read: $48,245$ μs, write: $48,444$ μs), while Multi-Paxos achieves the most balanced and lowest overall latencies (read: $31,977$ μs, write: $31,357$ μs). ABD shows an interesting asymmetric pattern with excellent write latency ($30,313$ μs) but higher read latency ($37,825$ μs). This asymmetry in ABD reflects its protocol design, which optimizes for write operations at the cost of slightly higher read complexity.

4.0.3 Network Communication Efficiency

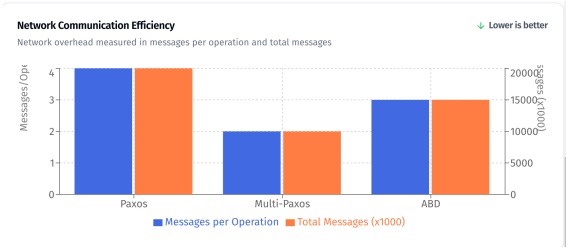

Figure 3: Network Communication Efficiency

The network efficiency analysis highlights the messaging overhead of each protocol. Basic Paxos requires 4 messages per operation, resulting in approximately 20,000 messages for 5,000 operations. Multi-Paxos achieves the best efficiency with 2 messages per

operation (10,000 total messages) after leader election. ABD maintains a middle ground with 3 messages per operation on average (15,000 total messages). This metric directly correlates with the observed throughput performance, demonstrating how message complexity impacts overall system efficiency.

4.0.4 Performance Over Time

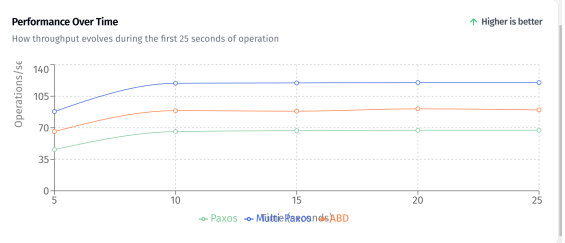

Figure 4: Performance Over Time

Time-series analysis reveals interesting stability patterns. Multi-Paxos shows the most stable performance, maintaining consistent throughput around 120 ops/sec during peak operation. ABD demonstrates good stability but with lower peak throughput (around 90 ops/sec), while basic Paxos stabilizes at the lowest level (around 67 ops/sec). All protocols show some initial ramp-up period, with Multi-Paxos achieving optimal performance fastest.

4.0.5 Summary

The experimental results demonstrate that both Multi-Paxos and ABD offer significant improvements over basic Paxos, with Multi-Paxos showing slightly better overall characteristics for balanced workloads. ABD proves particularly efficient for write operations, making it a strong candidate for write-heavy workloads.

5 Limitations and Future Work

5.1 Paxos

Our current implementation, while functional, has several areas for potential improvement:

Operation Recovery After Leadership Change: Currently, when a new leader is elected, pending operations at the old leader are lost. A more robust implementation would include a mechanism to transfer pending operations to the new leader. This could be achieved by having replicas resend their pending operations to the new leader after receiving a LeaderDecidedNotification.

Automatic Failure Detection and Cleanup: While our implementation handles leader failure through the heartbeat mechanism, it lacks automatic cleanup of crashed replicas from the membership. Implementing this would require:

- A failure detection mechanism similar to the leader heartbeat
- A consensus round to remove failed replicas
- State cleanup for removed replicas

Operation Processing Optimization: Our current implementation handles operation requests and their execution in the same thread/context - when a request arrives, the handler both processes the request and, if appropriate, executes the propose operation immediately. We attempted to separate these concerns by having request handlers only validate the request and add it to a pendingOperations queue, with a separate periodic timer responsible for processing this queue and executing the propose operations. However, this approach resulted in significant performance degradation compared to the immediate execution model, leading us to maintain the coupled implementation. The cause of this performance issue with the decoupled approach warrants further investigation.

Code and Data Structure Optimization: The current implementation has several redundant data structures and classes that could be consolidated:

- Merge similar message types (PaxosMessage variants)
- Unify notification structures across protocols
- Reduce code duplication in message handlers
- Simplify state tracking across Agreement and SMR protocols

Additional Improvements: Several other enhancements could be made to the system:

- Implement state transfer optimization to reduce the cost of bringing new replicas up-to-date
- Implement snapshotting for faster recovery
- Optimize the communication patterns between replicas

5.2 ABD

Implementation Issues: When testing the ABD implementation with heavier workloads, one of the replicas would stop replying to the application, however, as far as we tested, it would keep on participating in the quorum decisions. Additionally, we've noticed that disabling the logs and slightly increasing the interval from which pendingOperations would be executed would improve it's performance, which led us to believe that the underlying

problem could some synchronization issue that would result in a memory race (even if we used Concurrent Data Structures).

Timers: In hindsight, the use of timers to execute pending operations creates multiple synchronization issues, as future work, trying an implementation that didn't relied on timers such as creating threads once a request was received.

Membership Management: We also noticed that when introducing automatic membership management, the system would struggle even more with additional workload. Since it was also implemented with a timer, we also believe that it could be some synchronization issue.

Membership Algorithm: The membership algorithm could be stronger, for instance when removing a replica, replicas other than the one proposing the removal, don't explicitly wait for a quorum to remove the unresponsive replica.

6 Conclusion

Our implementation of Multi-Paxos based State Machine Replication demonstrates the practical viability of achieving consistent replication in distributed systems. The system's architecture, divided into an Agreement protocol for consensus and a State Machine protocol for state management, effectively handles both normal operations and membership changes while maintaining consistency. The experimental evaluation revealed several key insights about consensus protocols:

- Multi-Paxos significantly outperforms basic Paxos by reducing message complexity when a stable leader exists, improving overall system throughput by approximately 40
- Message efficiency is substantially improved in Multi-Paxos, requiring only 2 messages per operation after leader election compared to 4 messages in basic Paxos
- While ABD showed excellent write performance, Multi-Paxos demonstrated better balance between read and write operations, making it more suitable for mixed workloads

A ABD-based SMR

Algorithm 1 Key-Value Store Replication Using Majority Quorums (State & Init)

Interface:

Requests:

`write`(k, v) where k is the key, v is the value

`read` where k is the key

Indications:

`writeOK`(k)

`readOK`(k, v)

State:

$replicas$ set containing all replicas

$myself$ unique index of this replica

tag set with timestamps - pair (sequenceNumb, process) - per keys

val set of values for every key k, initially $\perp$

$opSeq$ unique sequence number for operations at this process

$answers$ set containing reply messages

$pending$ pending value to be written

Upon Init() do:

If isInitialMembership():

$replicas \leftarrow \Pi$

Else:

$replicas \leftarrow \perp$

$myself \leftarrow$ `mySequence`(replicas)

$tag \leftarrow \{\}$

$val \leftarrow \{\}$

$opSeq \leftarrow 0$

$pending \leftarrow \perp$

Algorithm 2 Key-Value Store Replication Using Majority Quorums (Write & Read)

Upon write(k, v) **do**:
  If $replicas \neq \perp$:
    $pending \leftarrow v$
    $opSeq \leftarrow opSeq + 1$
    $answers \leftarrow \{\}$
    Trigger bebBcastSend($\langle$READ-TAG, $opSeq$, $k\rangle$)

Upon bebBcastDeliver(READ-TAG, id**,** $k\ \rangle$**,** p**) do:**
  If $replicas \neq \perp$:
    Send(READ-TAG-REPLY, p, id, $tag[k]$)

Upon Receive($\langle$ **READ-TAG-REPLY,** id**,** $tag\rangle$**,** p **) do:**
  If $replicas \neq \perp$:
    If $opSeq = id$ **Then**
      $answers \leftarrow answers \cup \{\langle$ READ-TAG-REPLY, id, $tag\ \rangle\}$
      If $|answers| = \frac{\#\Pi+1}{2} + 1$ **Then**
        $new\text{-}tag \leftarrow \texttt{maxSQTag}(answers)$
        $opSeq \leftarrow opSeq + 1$
        $answers \leftarrow \{\}$
        Trigger bebBcastSend($\langle$ WRITE, $opSeq$, k, $\langle new\text{-}tag + 1, myself\rangle$, $pending\ \rangle$)
        $pending \leftarrow \perp$

Upon bebBcastDeliver($\langle$ **WRITE,** id**,** k**,** new**-**tag**,** $v\ \rangle$**,** p**) do:**
  If $replicas \neq \perp$:
    If $new\text{-}tag > tag[k]$ **Then**
      $tag[k] \leftarrow new\text{-}tag$
      $val[k] \leftarrow v$
    Send(ACK, p, id, k)

Upon Receive(ACK, p**,** id**,** k**) do:**
  If $replicas \neq \perp$:
    If $opSeq = id$ **Then**
      $answers \leftarrow answers \cup \{\langle$ ACK, $id\ \rangle\}$
      If $|answers| = \frac{N+1}{2} + 1$ **Then**
        $answers \leftarrow \{\}$
        If $pending = \perp$ **Then**
          Trigger WriteOK(k)
        Else:
          Trigger ReadOK(k, $pending$)

Upon read(k) **do**:
  If $replicas \neq \perp$:
    $opSeq \leftarrow opSeq + 1$
    $answers \leftarrow \{\}$
    Trigger bebBcastSend($\langle$READ, $opSeq$, $k\rangle$)

Upon bebBcastDeliver ($\langle$**READ,** id**,** $k\rangle$**,** p**) do:**
  If $pending \neq \perp$:
    Send(READ-REPLY, p, id, k, $tag[k]$, $val[k]$)

Upon Receive($\langle$**READ-REPLY,** id**,** k**,** tag**,** $v\rangle$**,** p**) do:**
  If $replicas \neq \perp$:
    If $opSeq = id$ **Then**
      $answers \leftarrow answers \cup \{\langle$READ-REPLY, id, k, tag, $v\rangle\}$
      If $|answers| = \frac{\#\Pi}{2} + 1$ **Then**
        $tag \leftarrow \texttt{maxTag}(answers)$
        $pending \leftarrow \texttt{valueOfMaxTag}(answers)$
        $opSeq \leftarrow opSeq + 1$
        $answers \leftarrow \{\}$
        Trigger bebBcastSend($\langle$WRITE, $opSeq$, k, tag, $pending\rangle$)

Algorithm 3 Membership Management - State, Init() & Timer

Interface:

Requests:

`ChangeMembership(` r , ACTION) //where r is the replica identifier and ACTION = {ADD, REMOVE}

State:

$replicas$ set containing all replicas
$currentlyAlive$ set containing replicas currently active between timers
$pendingMembership$ replica waiting to join
$myself$ unique index of this replica
$membershipTag$ set with timestamps - pair (sequenceNumb, process)
$opSeq$ unique sequence number for operations at this process
$answers$ set containing reply messages
$pending$ pending value to be written
T period between healthchecks

Upon Init() do:

If ISINITIALMEMBERSHIP():
    $replicas \leftarrow \Pi$
Else:
    $replicas \leftarrow \perp$
$currentlyAlive \leftarrow replicas$
$myself \leftarrow$ `mySequence`(replicas)
$tag \leftarrow \{\}$
$val \leftarrow \{\}$
$opSeq \leftarrow 0$
$pending \leftarrow \perp$
Setup Periodic Timer HealthCheck(T)

Upon Timer HealthCheck **do:**

If $\#replicas \neq \#currentlyAlive$ **Then**
    Trigger bebBcastSend(READ-TAG-MEMBERSHIP, $currentlyAlive, \langle opSeq, REMOVE \rangle$)
Else
    Trigger bebBcastSend(Ping, $replicas$)
$currentlyAlive \leftarrow \{\}$

Upon Ping(p) **do:**

    Send(Pong, p)

Upon Pong(p) **do:**

    $currentlyAlive \leftarrow currentlyAlive \cup p$

Algorithm 4 Membership Management - Join & Remove

Upon ChangeMembership(r, ADD) **do:**
  If $replicas \neq \perp$:
    $pending \leftarrow r$
    $pendingMembership \leftarrow r$
    $opSeq \leftarrow opSeq + 1$
    $answers \leftarrow \{\}$
    Trigger bebBcastSend(READ-TAG-MEMBERSHIP, $replicas$, $\langle opSeq, ADD \rangle$)

Upon bebBcastDeliver(READ-TAG-MEMBERSHIP, id**, ACTION**$\rangle$**,** p**) do:**
  If $replicas \neq \perp$:
    Send(READ-TAG-REPLY-MEMBERSHIP, p $\langle id, membershipTag, ACTION \rangle$)

Upon Receive($\langle$ **READ-TAG-REPLY-MEMBERSHIP,** id**,** $membershipTag, ACTION \rangle$**,** p**) do:**
  If $replicas \neq \perp$:
    If $opSeq = id$ **Then**
      $answers \leftarrow answers \cup \{\langle$ READ-TAG-REPLY-MEMBERSHIP, id, $tag \rangle\}$
      If $|answers| = \frac{\#\Pi+1}{2} + 1$ **Then**
        $new\text{-}tag \leftarrow \texttt{maxSQTag}(answers)$
        $opSeq \leftarrow opSeq + 1$
        $answers \leftarrow \{\}$
        Trigger bebBcastSend(WRITEMEMBERSHIP, $replicas\langle, opSeq, \langle new\text{-}tag + 1, myself \rangle, pending,$ ACTION$\rangle$)
        $pending \leftarrow \perp$

Upon bebBcastDeliver($\langle$ **WRITEMEMBERSHIP,** id**,** $new\text{-}tag$**,** r**, ACTION** $\rangle$**,** p**) do:**
  If $new\text{-}tag > tag$ **Then**
    If $ACTION = ADD$ **Then**
      $membershipTag \leftarrow new\text{-}tag$
      $pendingMembership \leftarrow r$
    Else
      $replicas \leftarrow replicas \setminus \{pendingMembership\}$
  Send($\langle$ MEMBERSHIPACK, p, id $\rangle$)

Upon Receive($\langle$ **MEMBERSHIPACK,** p**,** id**, ACTION** $\rangle$**) do:**
  If $opSeq = id$ **Then**
    $answers \leftarrow answers \cup \{\langle$ ACK, id $\rangle\}$
    If $|answers| = \frac{N+1}{2} + 1$ **Then**
      $answers \leftarrow \{\}$
      If $pending = \perp$ **Then**
        If $ACTION = ADD$ **Then**
          Send($\langle$JOINREPLY, $pendingMembership$, $membershipTag$, $replicas$ $\rangle$)
        Else
          $replicas \leftarrow replicas \setminus \{pendingMembership\}$

Upon Receive($\langle$ **JOINREPLY,** p**,** $membershipTagReceived$**,** $replicasReceived$ $\rangle$**) do:**
  $replicas \leftarrow replicasReceived$
  $membershipTag \leftarrow membershipTagReceived$
  Trigger bebBcastSend(JOINED, $replicas, myself$)

Upon bebBcastDeliver(JOINED, p**) do:**
  $replicas \leftarrow replicas \cup \{p\}$
  $pendingMembership \leftarrow \perp$

A Multi-Paxos SMR

Algorithm 5 Multi-Paxos Agreement Protocol

State:
$self$ identity of this node
$membership$ list of nodes in the system
$currentLeader$ current leader node
$isLeader$ boolean indicating if this node is leader
$listOfProposes$ map of UUID to PaxosInstance
$decidedMessages$ map of UUID to LearnMessage

Upon proposeRequest($instance$, $opId$, $operation$) **do:**
if $opId \in listOfProposes$ **then**
    return
end if
if $\neg isLeader$ **then**
    $msg \leftarrow$ new OperationMessage($opId$, $self$, $instance$, $operation$)
    Send(msg, $currentLeader$)
    return
end if
$paxosInstance \leftarrow$ new PaxosInstance($opId$, 1, $membership$, $REGULAR$)
$paxosInstance.proposedValue \leftarrow$ new ProposedValue($REGULAR$, $null$, $operation$)
$listOfProposes[opId] \leftarrow paxosInstance$
if $currentLeader = null$ **then**
    // Need full Paxos round if no leader
    $msg \leftarrow$ new PrepareMessage($paxosInstance.id$, $paxosInstance.ballot$, $opId$, $REGULAR$)
else
    // Skip prepare phase if leader exists
    $msg \leftarrow$ new AcceptMessage($paxosInstance.id$, $paxosInstance.ballot$, $opId$)
end if
for all $p \in membership \wedge p \neq self$ **do**
    Send(msg, p)
end for

Upon Receive(PrepareMessage, $sender$) **do:**
$paxosInstance \leftarrow listOfProposes[msg.operationId]$
if $paxosInstance = null$ **then**
    $type \leftarrow$ msg.type = JOIN ? JOIN : REGULAR
    $paxosInstance \leftarrow$ new PaxosInstance($msg.id$, $msg.ballot$, $membership$, $type$)
    $listOfProposes[msg.operationId] \leftarrow paxosInstance$
end if
if $msg.ballot \geq paxosInstance.ballot$ **then**
    $paxosInstance.ballot \leftarrow msg.ballot$
    $paxosInstance.promises[sender] \leftarrow true$
    $promise \leftarrow$ new PromiseMessage($paxosInstance.id$, $msg.ballot$, $msg.operationId$, $msg.type$)
    Send($promise$, $sender$)
end if

Upon Receive(PromiseMessage, $sender$) **do:**
if $\neg isLeader$ **then**
    return
end if
$paxosInstance \leftarrow listOfProposes[msg.operationId]$
$paxosInstance.promises[sender] \leftarrow true$
if hasQuorum($paxosInstance.promises$) **then**
    $accept \leftarrow$ new AcceptMessage($paxosInstance.id$, $paxosInstance.ballot$, $msg.operationId$)
    for all $p \in membership \wedge p \neq self$ **do**
        Send($accept$, p)
    end for
end if

Upon Receive(AcceptMessage, $sender$) **do:**
$paxosInstance \leftarrow listOfProposes[msg.operationId]$
if $msg.ballot \geq paxosInstance.ballot$ **then**
    $paxosInstance.ballot \leftarrow msg.ballot$
    $paxosInstance.accepts[sender] \leftarrow true$

Algorithm 6 Multi-Paxos Leader Election and Membership Management

State:
$LEADER\_TIMEOUT = 5000$ milliseconds
$receivedHeartbeat$ boolean flag for heartbeat reception

Upon Init($initialMembership$) **do:**
if $self = membership[0]$ **then**
$isLeader \leftarrow true$
$currentLeader \leftarrow self$
Setup Periodic Timer HeartbeatTimer($LEADER\_TIMEOUT/2$)
else
$isLeader \leftarrow false$
$currentLeader \leftarrow membership[0]$
Setup Periodic Timer LeaderTimer($LEADER\_TIMEOUT$)
end if

Upon Timer HeartbeatTimer **do:**
if $isLeader$ **then**
$heartbeat \leftarrow$ new HeartbeatMessage()
for all $p \in membership \land p \neq self$ **do**
Send($heartbeat$, p)
end for
end if

Upon Timer LeaderTimer **do:**
if $\neg isLeader \land \neg receivedHeartbeat$ **then**
StartLeaderElection
end if
$receivedHeartbeat \leftarrow false$

Upon joinRequest($instance$, $opId$, $node$) **do:**
if $\neg isLeader$ **then**
$msg \leftarrow$ new JoinMessage($opId$, $node$)
Send(msg, $currentLeader$)
return
end if
if $|membership| < 3$ **then**
$membership \leftarrow membership \cup \{node\}$
$stateTransfer \leftarrow$ new StateTransferMessage($appState$, $membership$)
Send($stateTransfer$, $node$)
return
end if
$proposeRequest \leftarrow$ new ProposeRequest($instance$, $opId$, $JOIN$, $node$)
Trigger proposeRequest

Procedure StartLeaderElection()
$currentLeader \leftarrow \perp$
$electionId \leftarrow$ generateUUID()
$ballot \leftarrow$ generateNewBallot()
$paxosInstance \leftarrow$ new PaxosInstance($electionId$, $ballot$, $membership$, $LEADER\_ELECTION$)
$paxosInstance.proposedValue \leftarrow$ new ProposedValue($LEADER\_ELECTION$, $self$, $null$)
$msg \leftarrow$ new PrepareMessage($electionId$, $ballot$, $electionId$, $LEADER\_ELECTION$)
for all $p \in membership$ **do**
Send(msg, p)
end for

Algorithm 7 State Machine Protocol

State:
$self$ current node identity
$membership$ list of current members
$currentLeader$ current leader node
$isLeader$ boolean indicating if this node is leader
$ready$ boolean indicating readiness to process operations
$nextInstance$ next instance number for operations
$decided$ set of decided operations
$appState$ application state - key-value map

Upon orderRequest($opId$, $operation$) **do:**
if $\neg ready$ **then**
    return
end if
if $opId \in decided$ **then**
    return
end if
if $\neg isLeader$ **then**
    $msg \leftarrow$ new OperationMessage($opId$, $self$, $nextInstance$, $operation$)
    Send(msg, $currentLeader$)
    return
end if
$proposeRequest \leftarrow$ new ProposeRequest($nextInstance$, $opId$, $operation$)
$nextInstance \leftarrow nextInstance + 1$
Trigger proposeRequest

Upon nodeDecidedNotification($opId$, $type$, $node$) **do:**
if $node \in membership$ **then**
    return
end if
$membership \leftarrow membership \cup \{node\}$
if $isLeader$ **then**
    $stateTransfer \leftarrow$ new StateTransferMessage($appState$, $membership$)
    Send($stateTransfer$, $node$)
end if

Upon decisionNotification($opId$, $decision$, $membership$, $payload$) **do:**
if $opId \in decided$ **then**
    return
end if
if $decision = COMMIT$ **then**
    $nextInstance \leftarrow nextInstance + 1$
    $decided \leftarrow decided \cup \{opId\}$
    if $payload \neq null$ **then**
        ExecuteOperation($opId$, $payload$)
    end if
end if

Upon leaderDecidedNotification($opId$, $newLeader$) **do:**
$currentLeader \leftarrow newLeader$
$isLeader \leftarrow newLeader = self$
if $isLeader$ **then**
    $ready \leftarrow true$
else
    Open Connection($currentLeader$)
end if